

Decoding Number Systems

A visual blueprint to Base 8, Base 16,
and digital translation.

```
10110010101101100010101101000010000110101
11001011010001101001001011000101001101101
10101000110110101010101011101010110111100
11000010100110010101100100101101101101001
0011001001001100100101010101101010110100
10110010101101100010101101000010000110101
11001011010001101001001011000101001101101
10101000110110101010101011101010110111100
11000010100110010101100100101101101101001
0011001001001100100101010101101010110100
```

```
1101001011011001001101
```

```
1001011010001
```

```
100111001000
```

```
101100101011
```

```
110010110100
```

```
011010001101
```

```
110000101001
```

```
100010011001
```

1A3₁₆

```
101100101011
```

```
110010110100
```

```
011101100101
```

```
101010101001
```

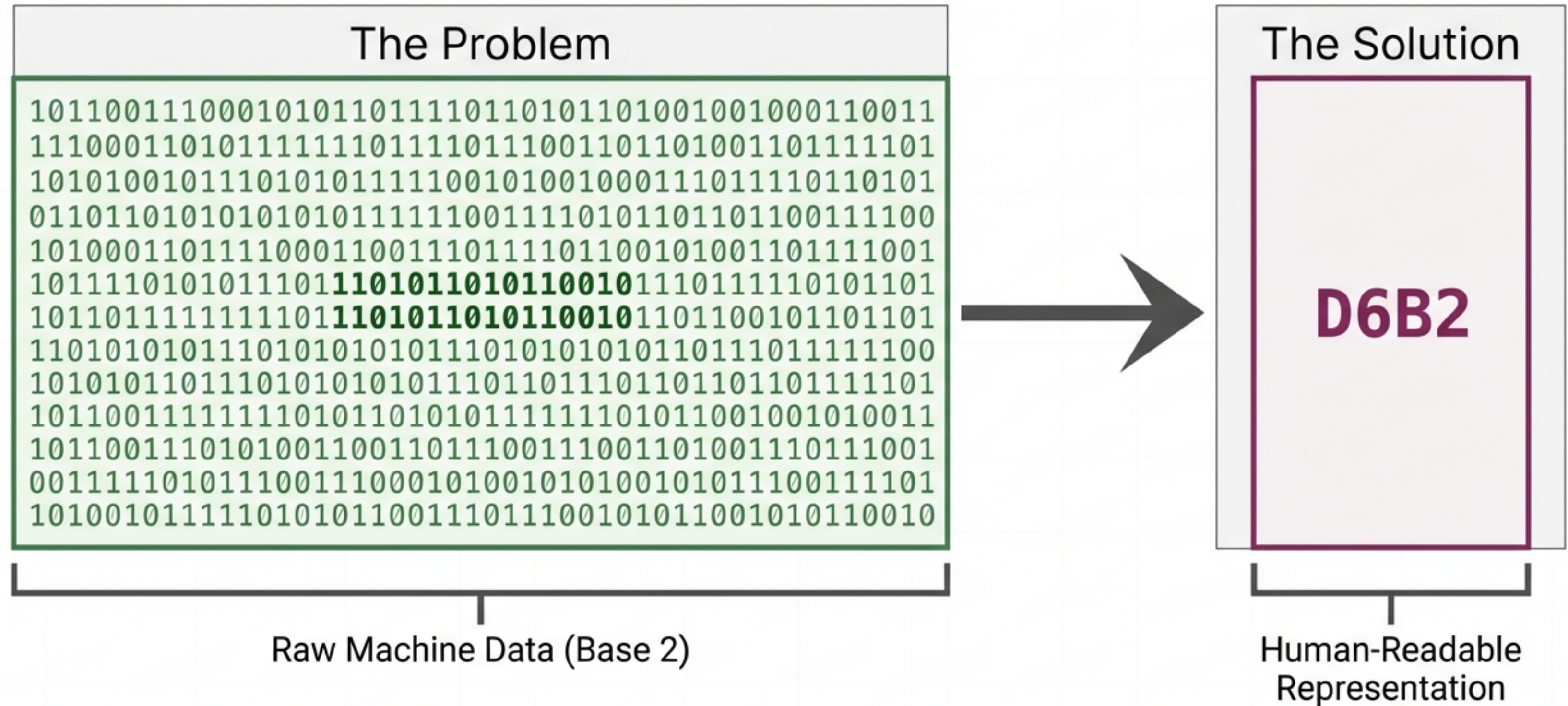
```
110000101001
```

```
001100100100
```

```
110010110101
```

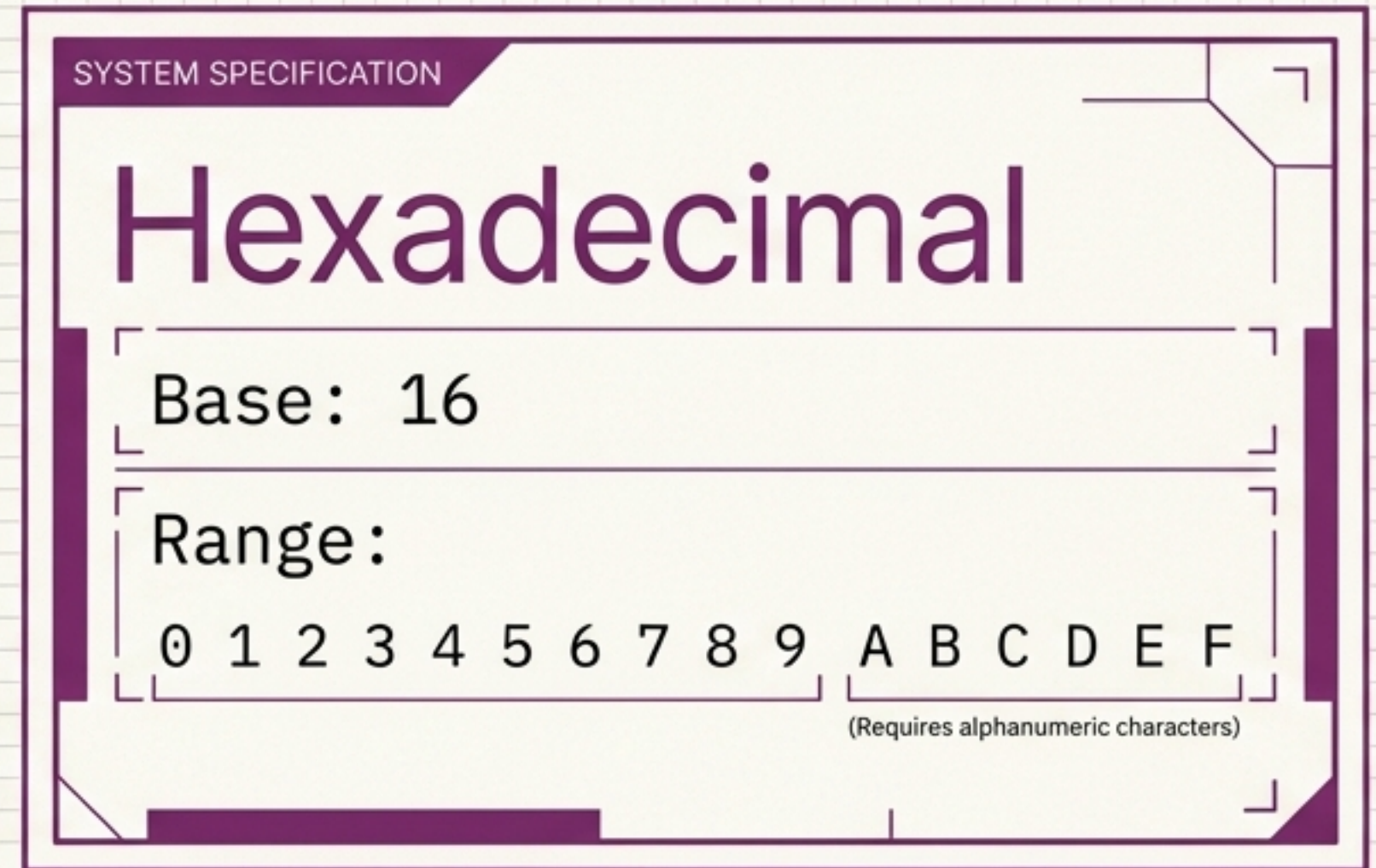
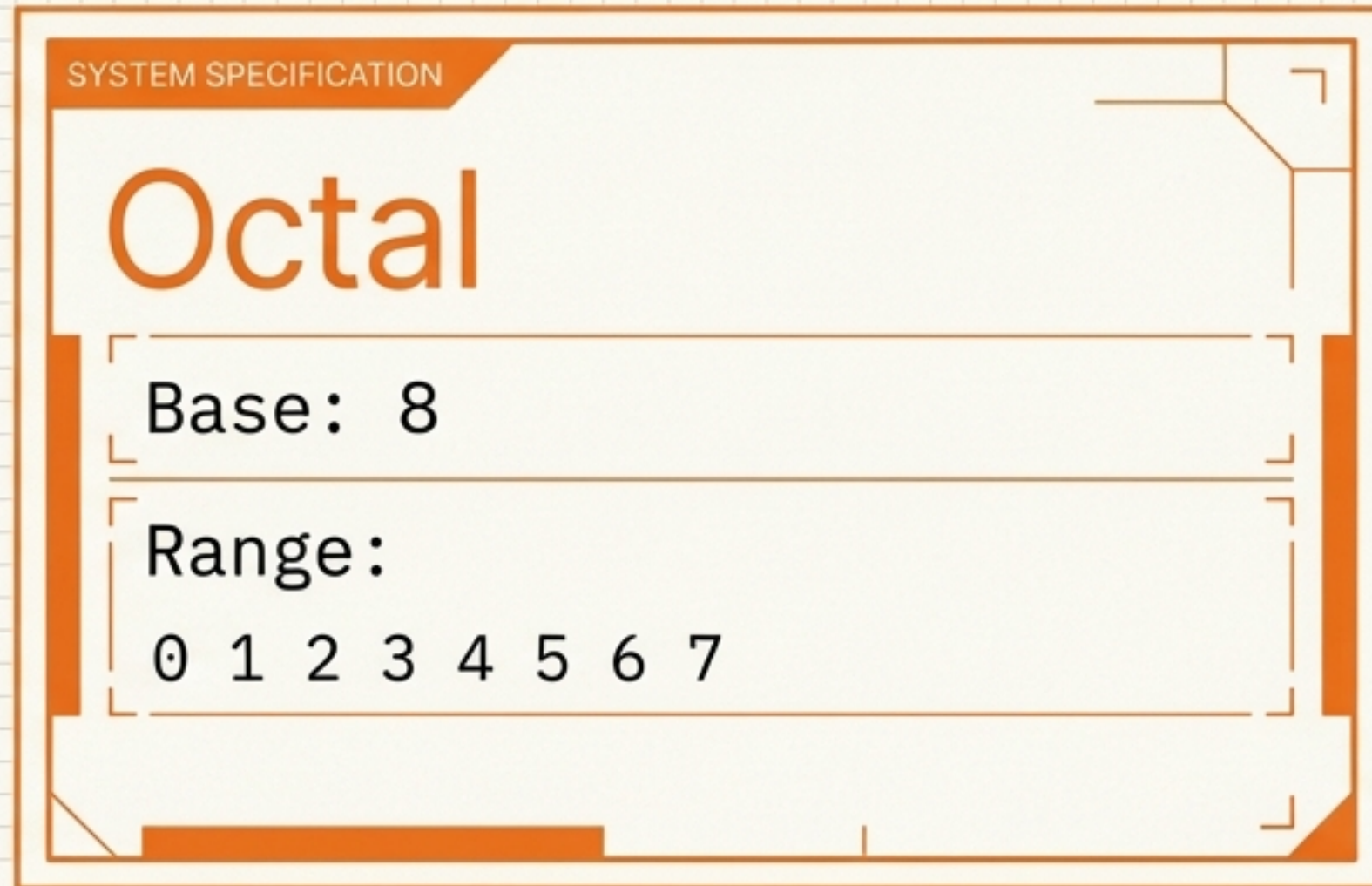
157₈

The hardware speaks binary, but we need compression.



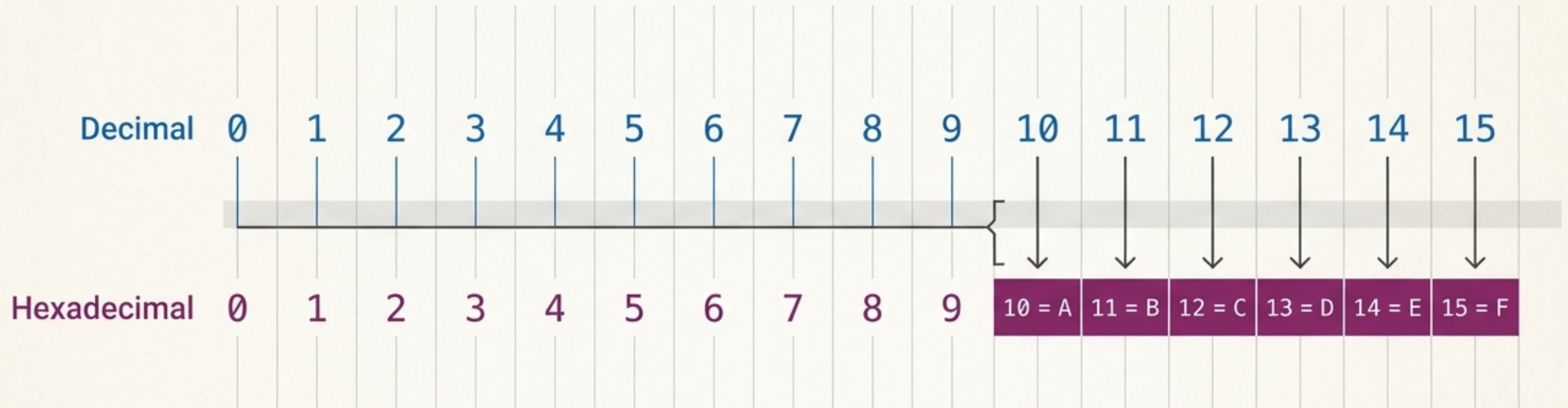
Modern computers process data exclusively in Binary (Base 2). Octal and Hexadecimal systems act as compact representations, compressing long binary strings into manageable chunks for programmers and interfaces.

Meet the compression languages.



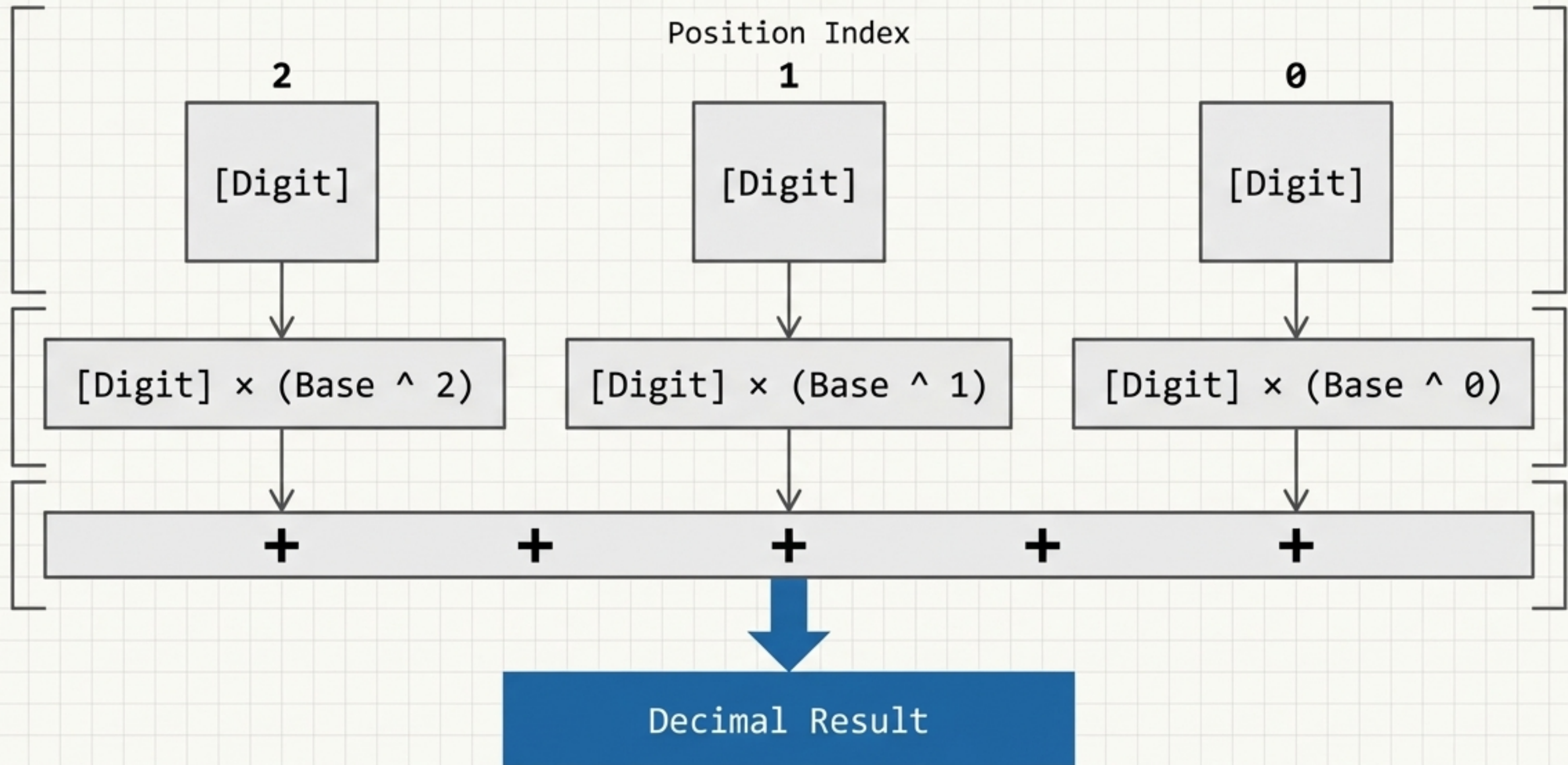
Both are positional number systems. The mathematical value of any digit is determined entirely by its placement from right to left.

Bridging the double-digit gap in Base 16.



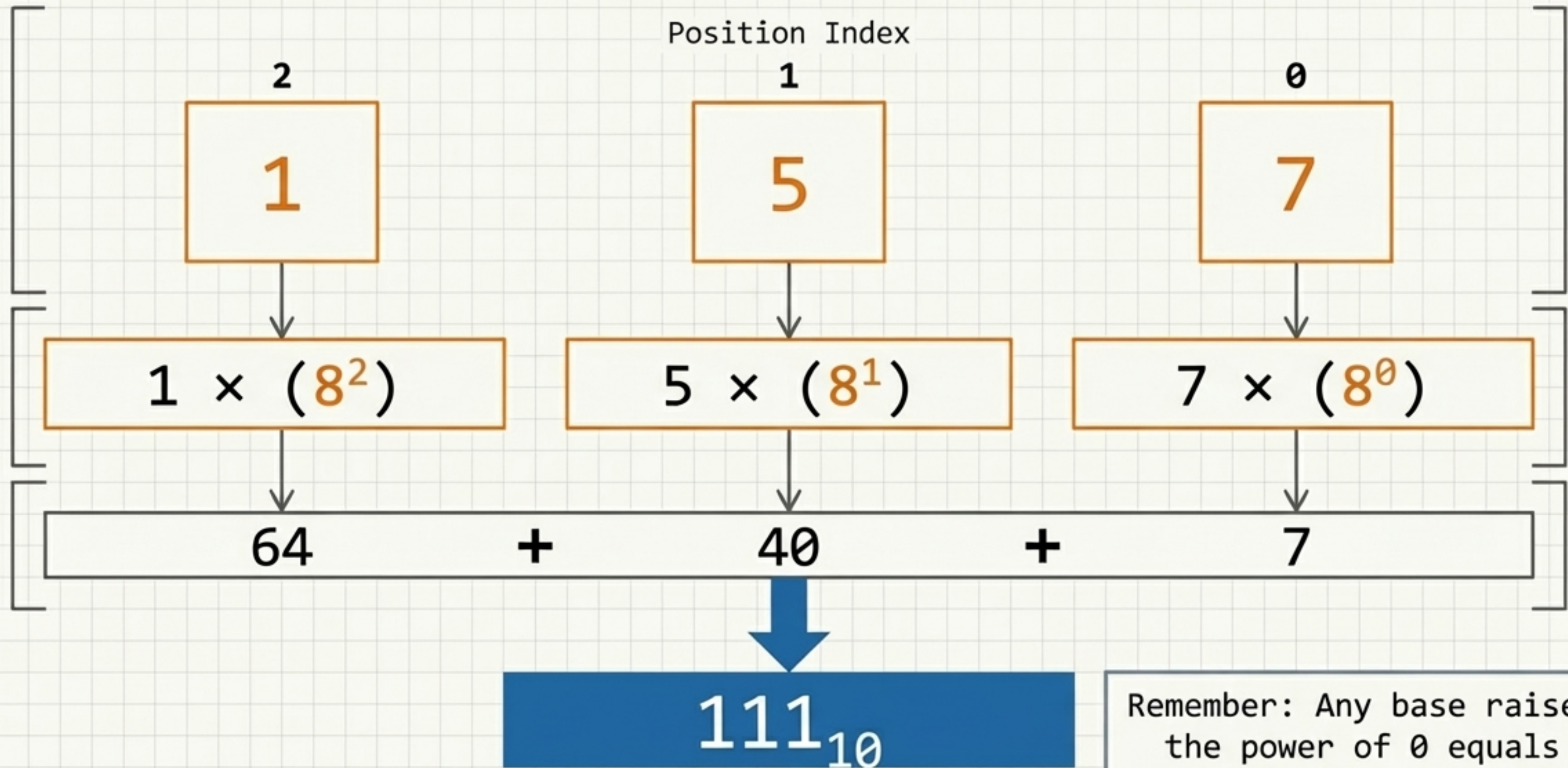
A single positional slot in Base 16 must hold a value up to 15. If we wrote '10', the computer would read it as two separate digits in two different slots. Letters provide a single-character solution.

Rule #1: The Power Multiplier (Converting to Decimal)



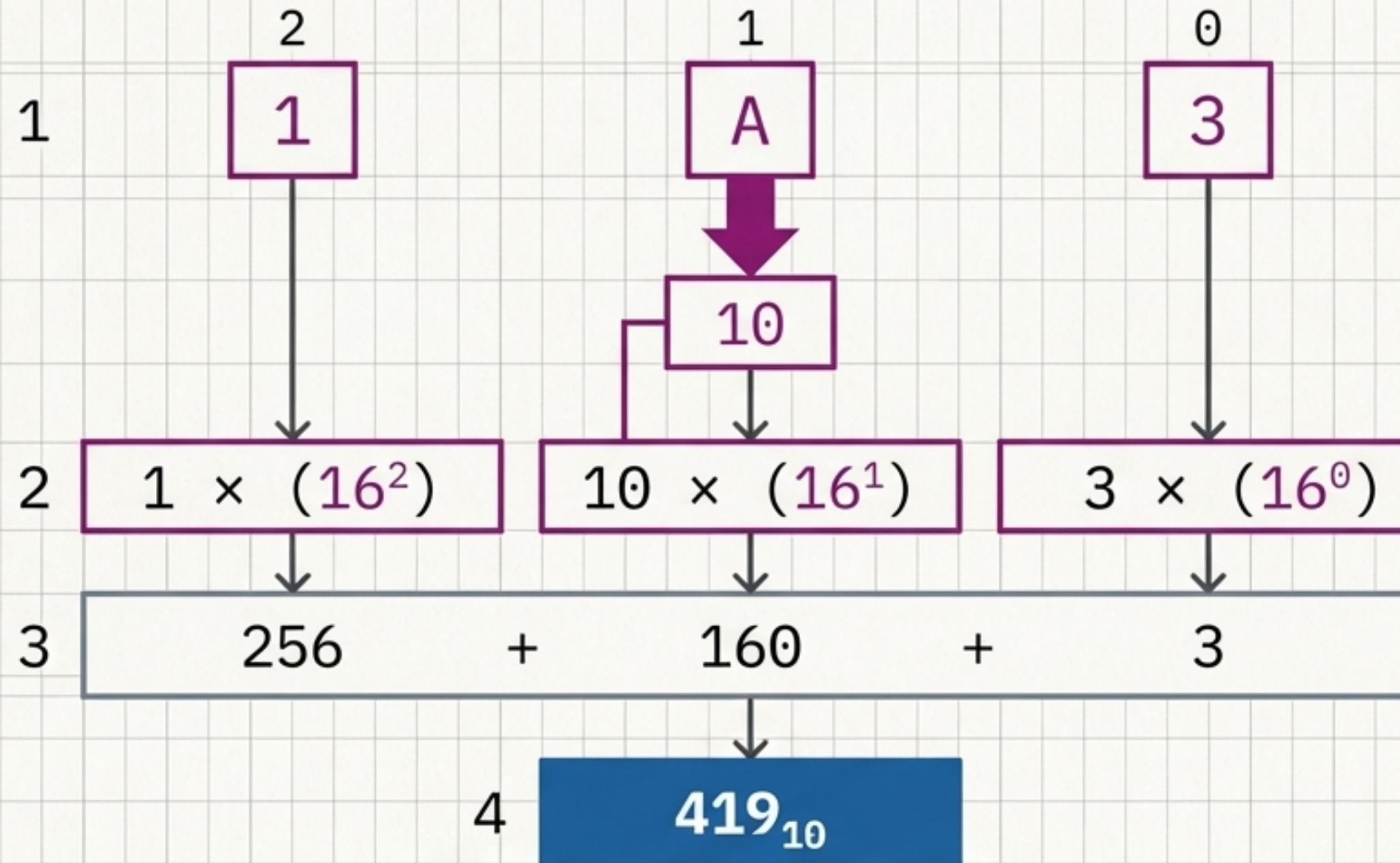
To translate any foreign system back into standard Decimal (Base 10), multiply each digit by its source base raised to the power of its position index, then sum the results.

Unpacking an Octal string into Decimal.



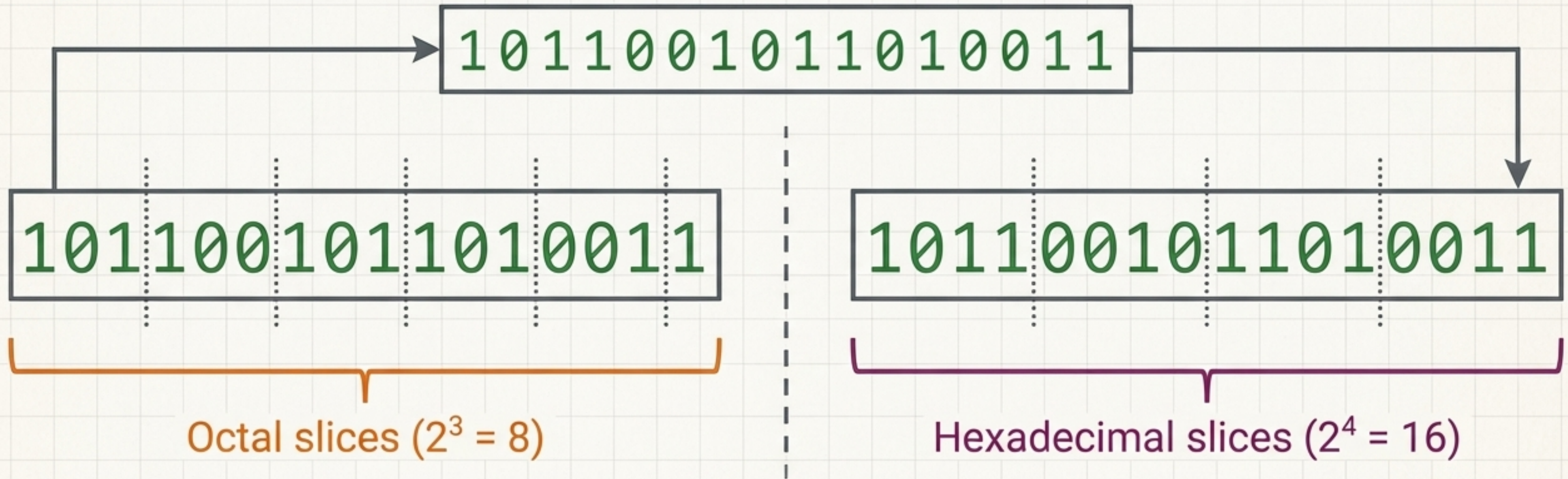
Remember: Any base raised to the power of 0 equals 1. Therefore, $7 \times (8^0)$ becomes $7 \times 1 = 7$.

Translating Hexadecimal letters before multiplication



Always convert alphabet characters back into their integer values (A = 10) before executing the base multiplication.

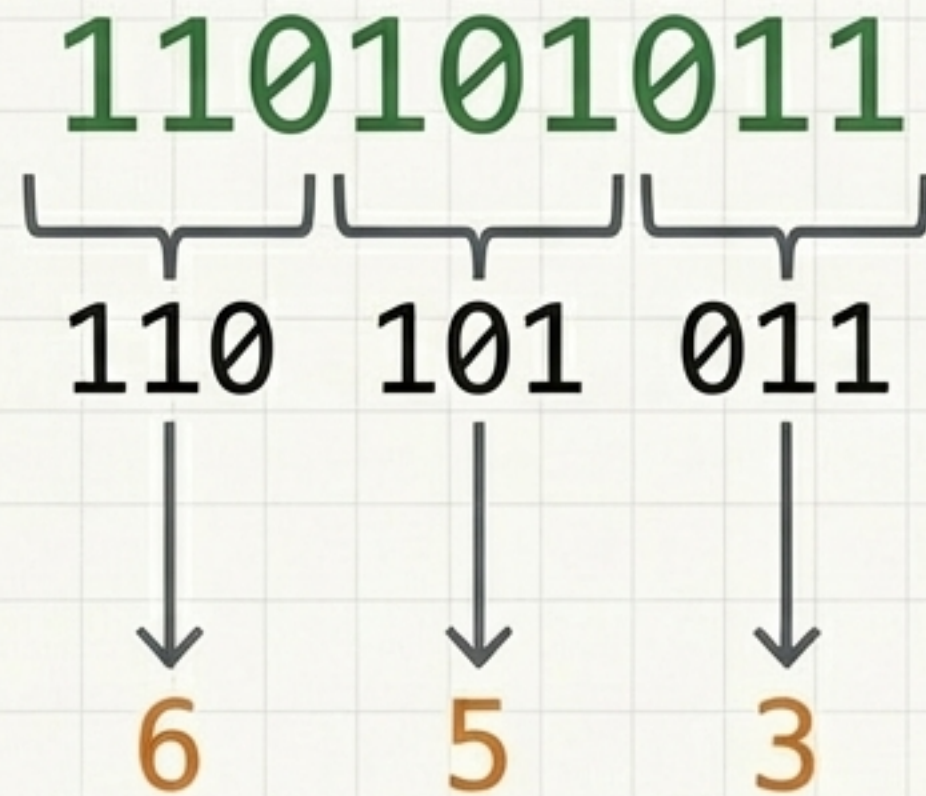
Rule #2: The Bit Bundler (Compressing Binary)



Because Base 8 and Base 16 are perfect powers of 2, we bypass Decimal entirely. We simply slice the binary string into specific chunk sizes and translate each chunk independently.

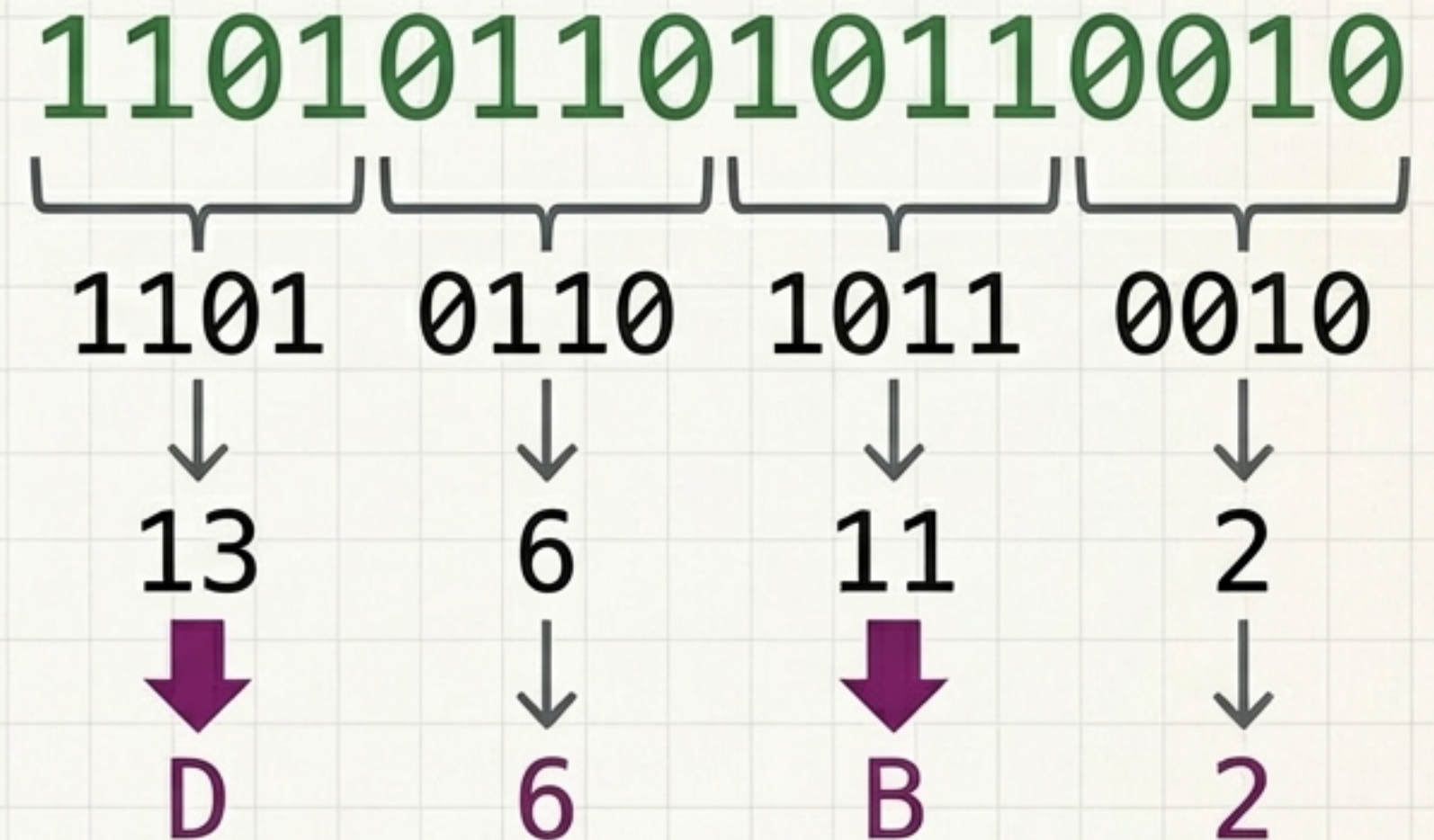
Slicing the tape: 3-bit vs 4-bit grouping.

Octal (Base 8) - 3-bit grouping



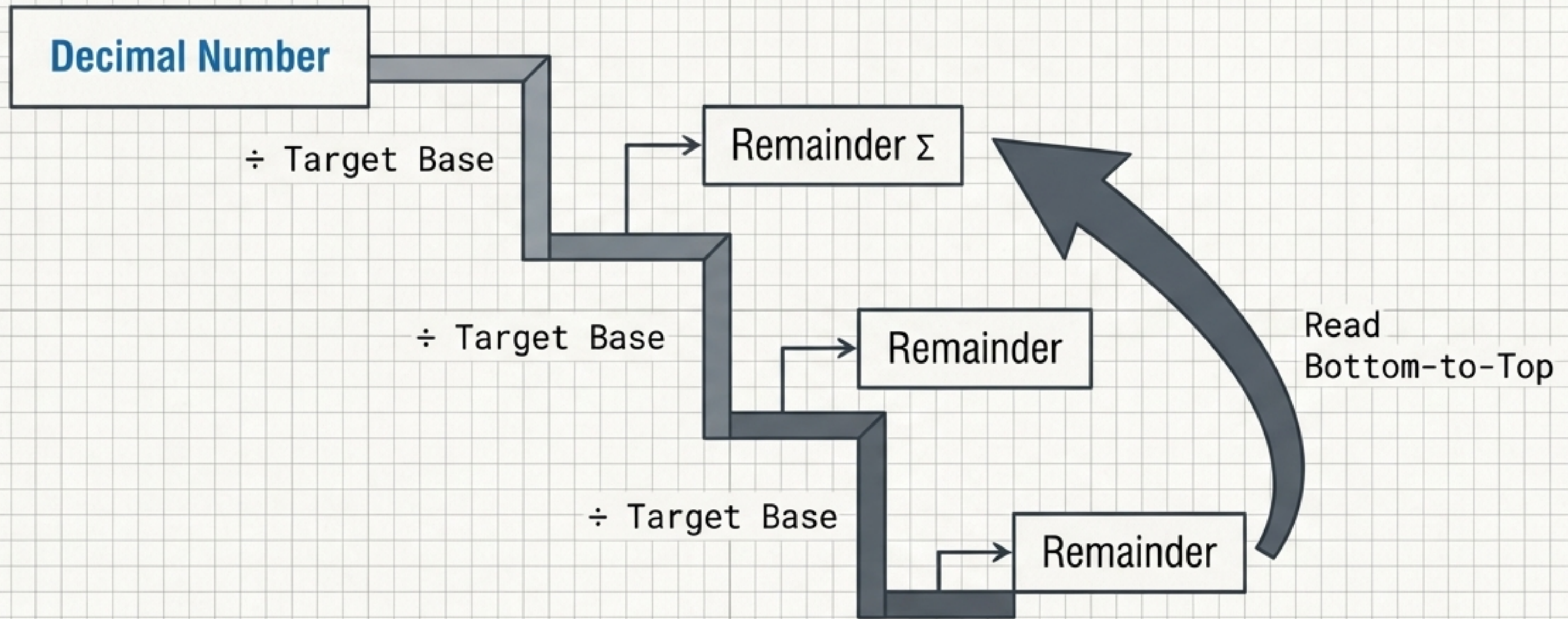
653₈

Hexadecimal (Base 16) - 4-bit grouping



D6B2₁₆

Rule #3: The Remainder Ladder (Converting from Decimal)



To convert a Decimal number into a new system, repeatedly divide by the target base until the quotient reaches zero. The remainders, read sequentially from bottom to top, form your new digital string.

Dividing into Octal format.

83

$$83 \div 8 = 10$$

Remainder 3

$$10 \div 8 = 1$$

Remainder 2

$$1 \div 8 = 0$$

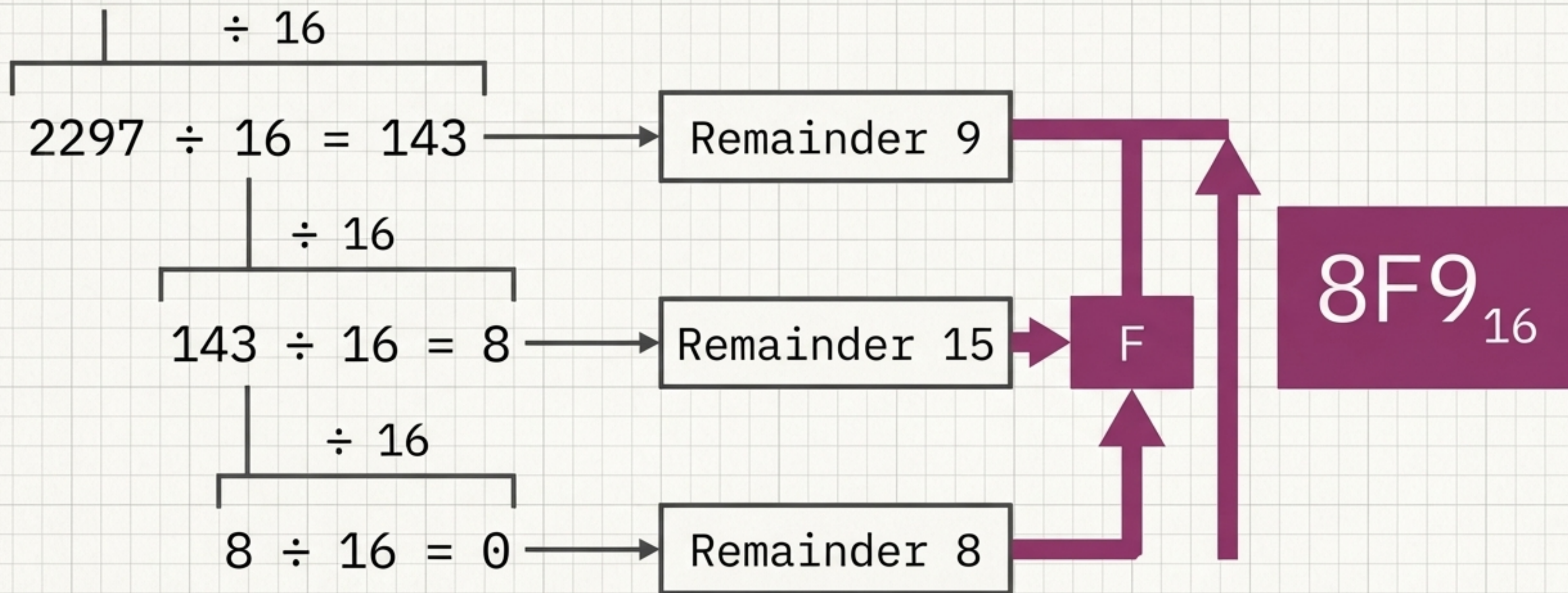
Remainder 1

Division stops completely
the moment the resulting
quotient hits 0.

123₈

Catching Hexadecimal alphabets in the remainder.

2297



The Digital Rosetta Stone

	Binary	Octal	Decimal	Hexadecimal
Base Number	Base 2	Base 8	Base 10	Base 16
Valid Digits	0-1	0-7	0-9	0-9, A-F
Binary Grouping Size	1 bit	3 bits	N/A	4 bits
Primary Role	Machine hardware logic	Compact binary representation	Human mathematics	High-density data encoding

Master these rules, and you can map and translate any layer of computing architecture.